APACHE STORM

DISTRIBUTED, FAULT-TOLERANCE REAL-TIME COMPUTATION

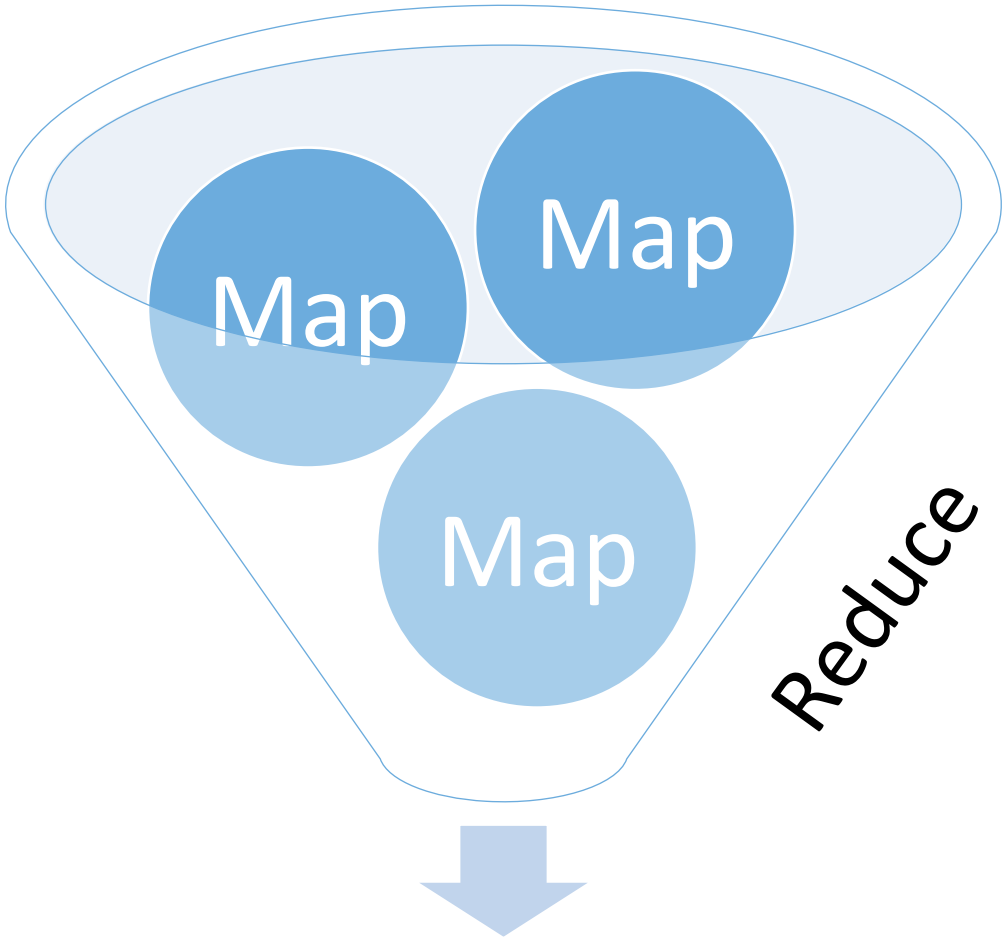
CA4022 ALESSANDRA MILEO

ALESSANDRA.MILEO@DCU.IE

Part 1: Introduction to Storm

MapReduce

- Works on Files
  - Fixed amounts of data
- Batch processing
  - Large amounts of data
  - Divide and Conquer
- Jobs run, then they stop
- Jobs can restart



Stream Data

- Real-time **data**
- Real-time **processing**

- A **Stream** is an unbounded sequence of tuples

https://www.flickr.com/photos/mabi/38307124/

Stream Processing

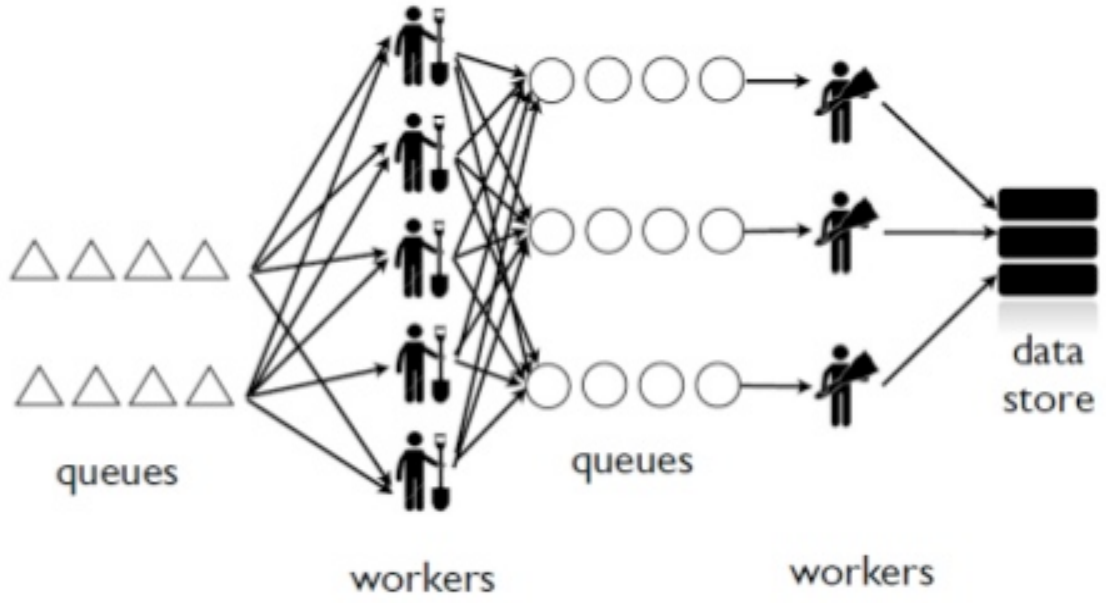
Streams need to be addressed differently to static data

- Infinite amounts of data
- No complete picture
- Respond to changes in the data flow
- Streams can be combined, split

Realtime processing before Storm

Queues-Workers Model

- ***Tedious*** *to configure*
- ***Scaling is painful***
  (queue partitioning, workers deploy, messaging reconfiguration)
- ***Operational overhead*** for worker failures and queue backup
- ***Poor fault-tolerance***
  (you are responsible for keeping workers and queue up)



Storm vs. Hadoop

Storm Framework

- Real-time computation
- Process infinite streams of data (infinite computation)
- Low latency
- Processes topologies

Hadoop Framework

- Batch computation
- Process a lot of data at once (finite computation)
- High latency
- Processes MapReduce jobs

Storm adoption

- Twitter: personalization and search
  - 200 nodes, 30 topologies, 50B msgs a day, avg latency < 50ms (2013)
- Yahoo: user events, feeds and application logs
- Spotify: recommendations, ads, monitoring
- … and more:
  - Alibaba, Cisco, Flickr, Netflix…

Apache Storm

- Horizontally scalable (+ machines, + parallelism, + messages/s)
- No data loss
- Extremely robust
- Fault-tolerant (runs forever or till you kill it)
- Programming language agnostic
- Usecases: stream processing, Distributed RPC, continuous computation

Apache Storm

Storm processing is organised into **Topologies**

Data comes from **Spouts**

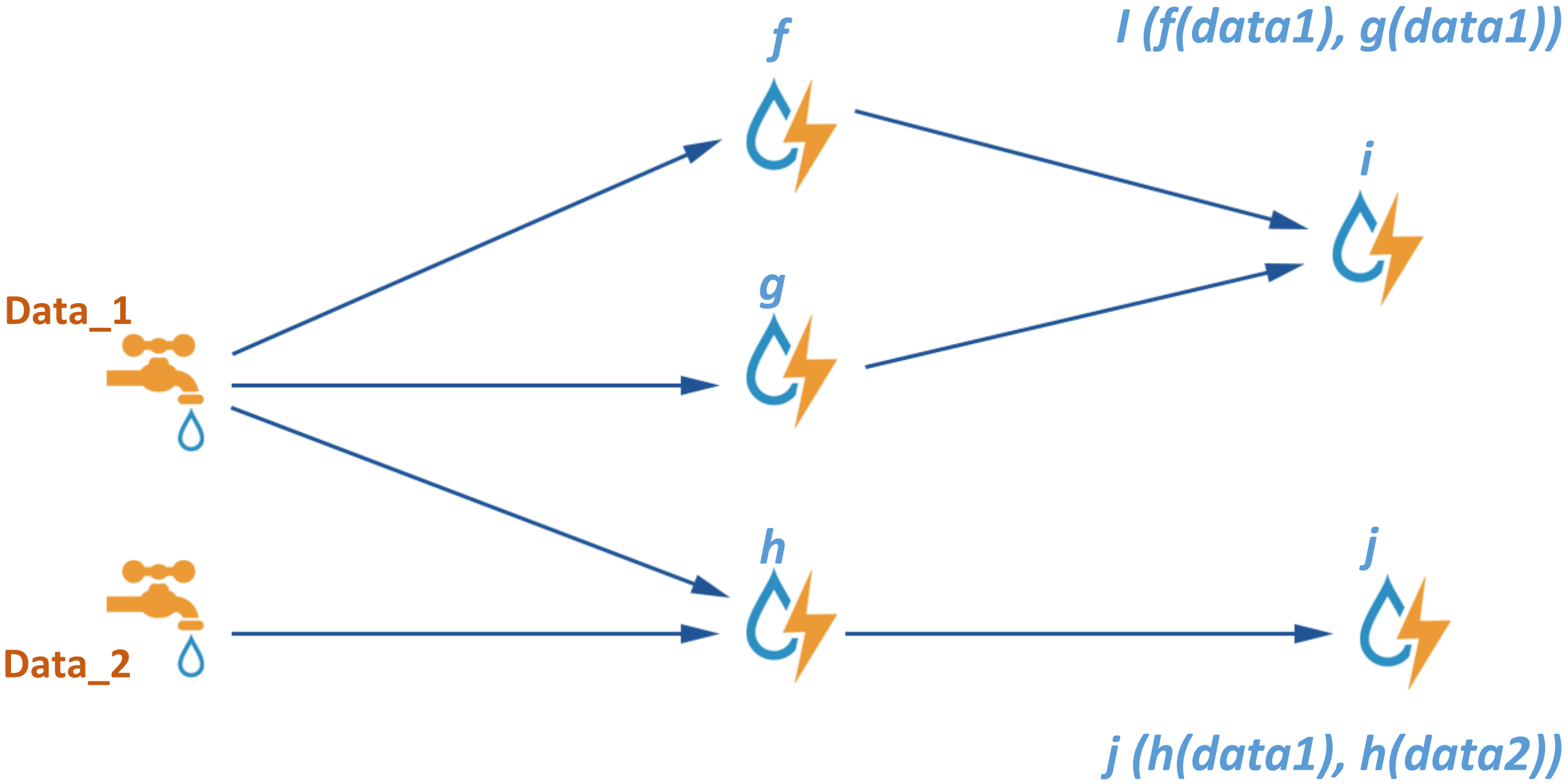
- Source of streams (as tuples)
- Message Queue (ZeroMQ, RabbitMQ, Apache Kafka)

Data is processed by **Bolts**

- Take multiple input streams
- Process unbounded streams (of tuples)
- Emit one or more output streams (of tuples)

Topology

- Analogous to a Job in MapReduce
- Directed Acyclic Graph (DAG) of Spouts and Bolts connecting data (spouts) and functions (bolts)
- Multiple layers of bolts
  - Each bolt performs a small task
- Nodes are parallelised
  - Concurrent execution of tasks


f
I (f(data1), g(data1))
i
Data_1
g
h
j
Data_2
j (h(data1), h(data2))

Spouts

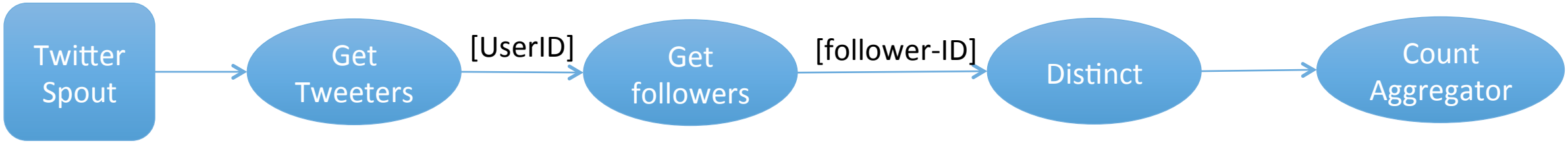
- Spouts emit one or more streams of data
  - One tuple at a time
- Spouts can be
  - **Reliable**: replaying failed tuples
  - **Unreliable**: letting failed tuples disappear
- Streams contain **Tuples**
  - Named list of values
  - Values can be objects
  - Default Types (e.g. Java primitive types) or custom objects in tuples

Bolts

- Bolts are the processing components
  - Filtering
  - Aggregation
  - Functions
  - Database Access
- Bolts subscribe to streams
- Complex stream transformations require multiple bolts

Example Topology: Reach

Reach is the number of unique people exposed to a specific URL on twitter



Structure of a Storm Cluster

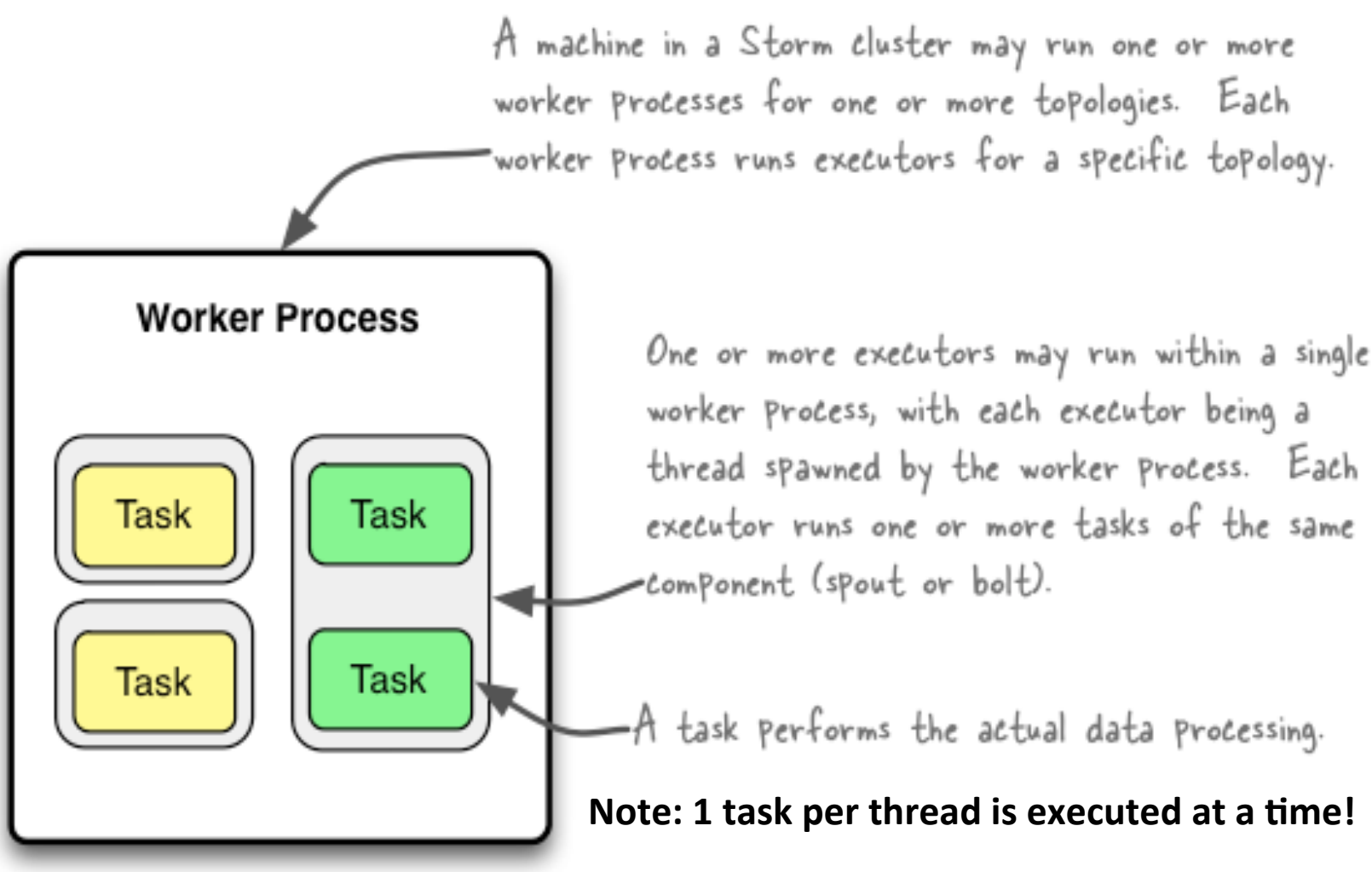
- Master node (Nimbus) similar to JobTracker – stateless and fail-fast
  - Distribute code around the cluster
  - Assigns tasks to workers
  - Monitor failures
- Worker node (Supervisor) similar to TaskTracker – stateless and fail-fast
  - Starts and stops worker processes
  - Listens for tasks
  - Executes part of the topology
- Zookeeper cluster
  - Coordination and distributed syncronisation
  - Maintains all states

Storm Parallelism: running a topology

3 Main entities:

- Workers (nodes)
- Executors (threads)
- Tasks

- Distributed mode
- Local mode
  - Debugging

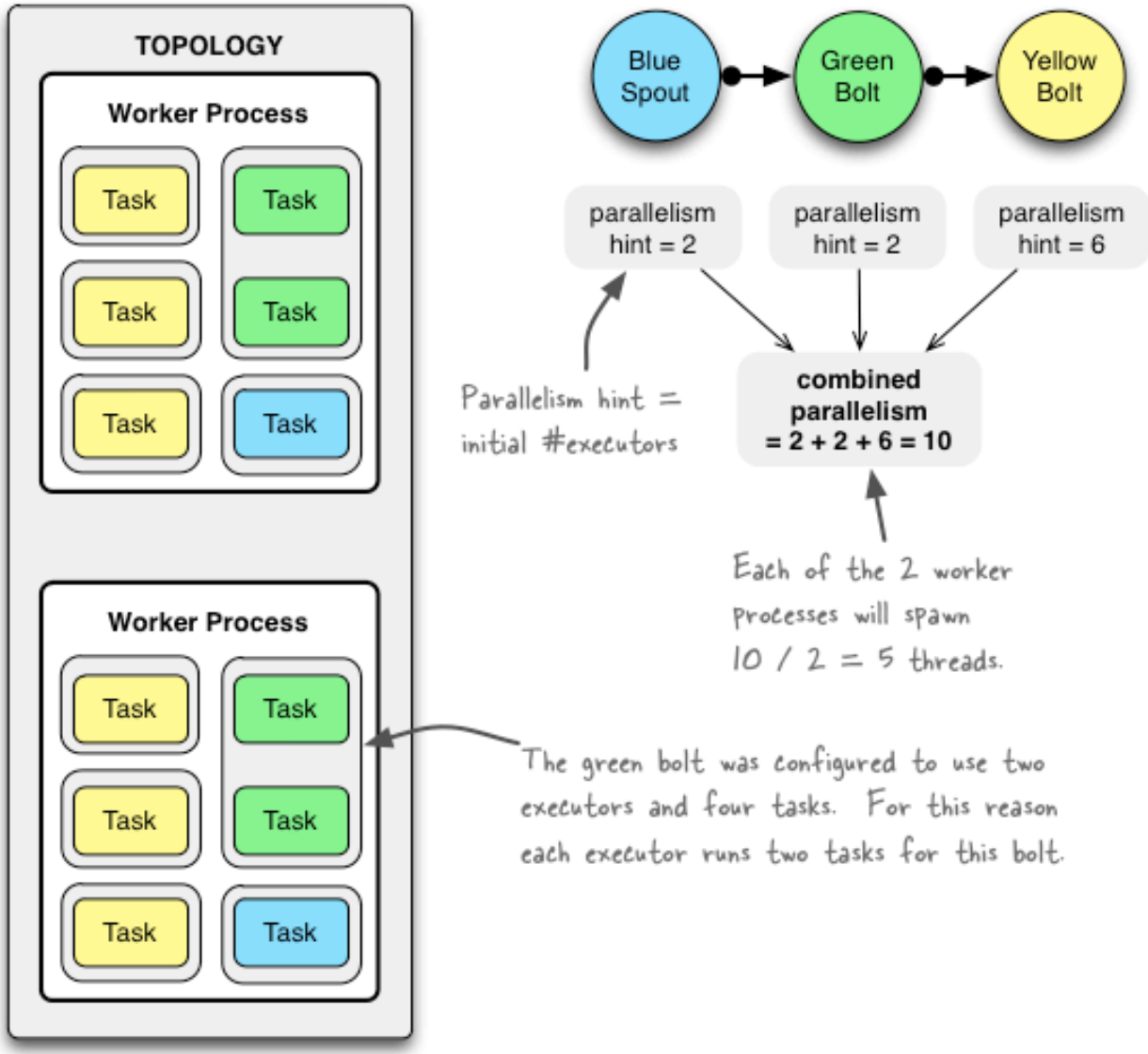


Note: 1 task per thread is executed at a time!

http://jansipke.nl/storm-in-pictures/

Storm Parallelism: running a topology

- One task per executors (default)
- Can configure #workers and #executors
- Load balancing:
  - Threads per worker
  - Tasks per thread
- Rebalancing (change executors/ workers without stopping the topology)
  - Storm Web UI
  - CLI (`storm rebalance`)

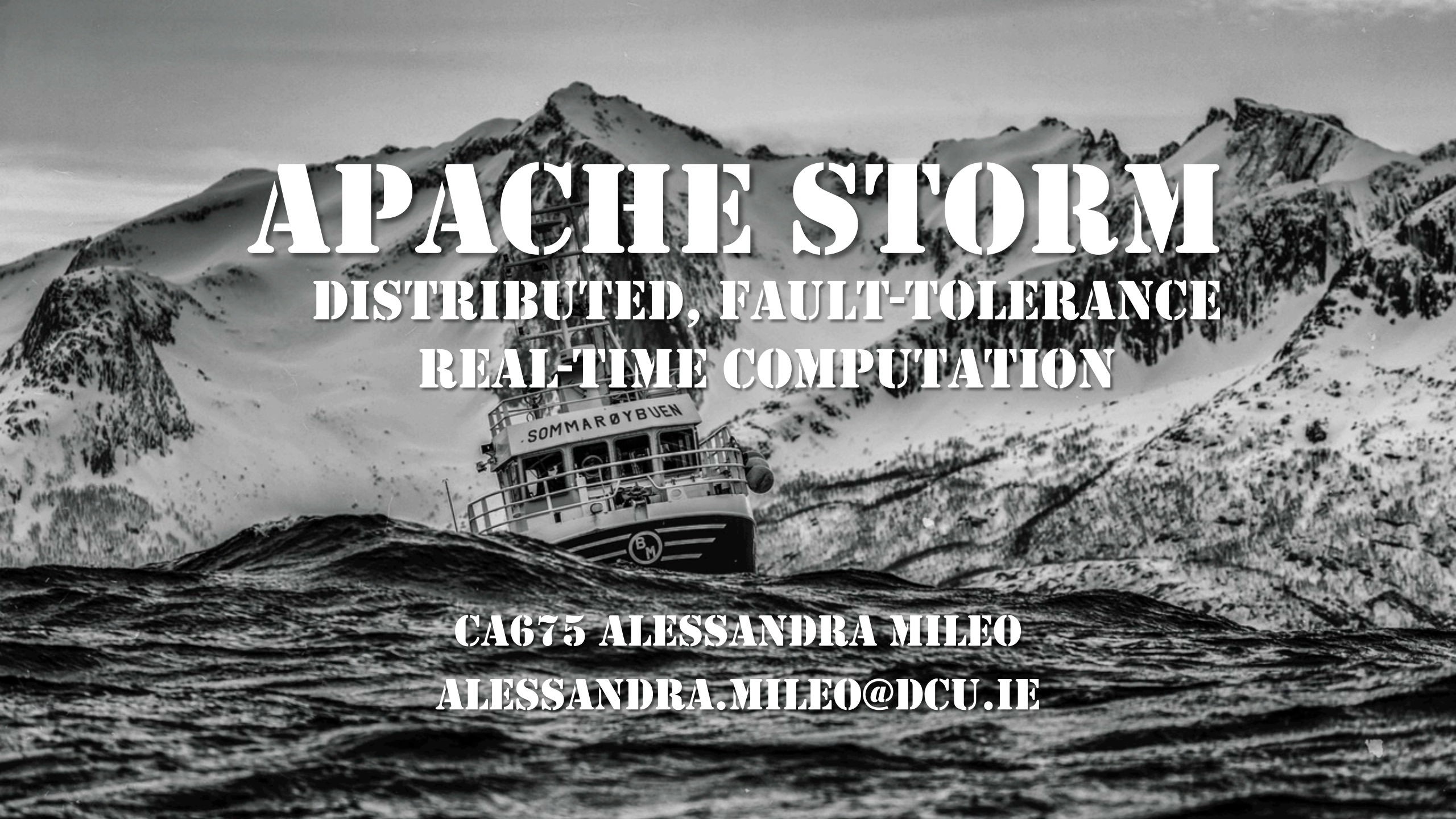


Stream Groupings

When a tuple is emitted, which task does it go to?

- Different Groupings for different tasks:
  - **Shuffle**: randomly assigned
  - **Fields**: based on the value of some tuple fields
  - **All**: sent to all tasks
  - **Global**: sent to a single task (task with lowest id)
  - **Special**: None/Direct/Local
- Declaring spouts/bolts also includes declaring their parallelism
- Connecting a Bolt to a Stream involves joining a Grouping

End of Part 1: Introduction to Storm



APACHE STORM

DISTRIBUTED, FAULT-TOLERANCE REAL-TIME COMPUTATION

CA675 ALESSANDRA MILEO

ALESSANDRA.MILEO@DCU.IE